Week 8, Lecture 1 WOD: Answer Key

Newton's Method by Hand (take-home)

NRES 746, Instructor Answer Key (not for student distribution)

Each item repeats the handout's question, then gives the answer, a short rationale, and mistakes to watch for. All numbers are computed in R when this file is knitted.

Plan about 8 minutes at the start of Wednesday's class to go over this, focusing on 3b and 3c. 3c leads straight into the lecture: the fixes students propose there (smaller steps, better starting values, fitting on the log scale) are what real optimizers do.

Part 1: Slope and curvature

1a.

> Find $\text{NLL}'(\lambda)$ and $\text{NLL}''(\lambda)$ for $\text{NLL}(\lambda) = 6\lambda - 18\log\lambda$. Then set $\text{NLL}'(\lambda) = 0$ and solve for the MLE, $\hat{\lambda}$.

Answer. $\text{NLL}'(\lambda) = 6 - \dfrac{18}{\lambda}$, $\text{NLL}''(\lambda) = \dfrac{18}{\lambda^2}$, and $\hat{\lambda} = 3$.

Why. $\frac{d}{d\lambda}\log\lambda = 1/\lambda$ and $\frac{d}{d\lambda}\lambda^{-1} = -\lambda^{-2}$ (the two minus signs cancel). $6 - 18/\lambda = 0$ gives $\lambda = 18/6 = 3$, the sample mean.

Watch for. A negative NLL'' (sign slip); the NLL has a minimum, so its curvature must be positive.

Part 2: Slopes without calculus

2a.

> Use the forward difference to estimate $\text{NLL}'(2)$ and compare it with the exact value from 1a. Then find the exact $\text{NLL}'(2.25)$. What do you notice?

Answer. Forward estimate: $(-1.4932 - (-0.4766))/0.5 = -2.0332$. Exact $\text{NLL}'(2) = -3$. Exact $\text{NLL}'(2.25) = 6 - 18/2.25 = -2$. The forward difference is a poor estimate of the slope at 2 but a very good estimate of the slope at 2.25.

Why. The forward difference is the slope of the chord from 2 to 2.5, and a chord's slope matches the curve's slope near the chord's middle (2.25), not at its left end. This is the idea 2b builds on.

Watch for. Sign errors from subtracting a negative number. Students who stop at "the estimate is off by about 1" without noticing the 2.25 match: ask where on the chord its slope best matches the curve.

2b.

> Curvature is the slope of the slope: how fast the slope changes per unit change in the parameter. The figure starts from a center point x and draws a backward difference (from x back to $x - \Delta$) and a forward difference (from x to $x + \Delta$). Combine these two slopes into a finite-difference approximation to $f''(x)$ that uses only $f(x - \Delta)$, $f(x)$, and $f(x + \Delta)$. (What you noticed in 2a may help you decide what to divide by.) Then use it to estimate $\text{NLL}''(2)$ and compare with the exact value.

Answer.

$$f''(x) \approx \frac{f(x+\Delta) - 2f(x) + f(x-\Delta)}{\Delta^2}$$

Estimate: $(-1.4932 + 0.9532 + 1.7016)/0.25 = 4.6464$. Exact: $18/4 = 4.5$.

Why. Forward difference minus backward difference is how much the slope changed; dividing by the distance over which it changed gives the change per unit of the parameter:

$$\frac{1}{\Delta}\left[\underbrace{\frac{f(x+\Delta) - f(x)}{\Delta}}_{\text{forward}} - \underbrace{\frac{f(x) - f(x-\Delta)}{\Delta}}_{\text{backward}}\right]$$

which simplifies to the formula above. The divisor is Δ, not 2Δ, because of 2a: each chord's slope belongs to its midpoint, so the forward slope sits at $x + \Delta/2$, the backward slope at $x - \Delta/2$, and those are Δ apart (the arrow in the figure).

Watch for.

- Dividing the change in slope by 2Δ, reasoning that the three points span 2Δ. Here that gives 2.32, half the right answer (as does dividing by Δ instead of Δ^2 overall). Send them back to 2a: where does each chord's slope belong?
- Treating the forward and backward differences as two estimates of the *same* slope at x and averaging them. That gives the first derivative (the centered slope, -3.1948), not the second.
- Getting the right formula by "guessing the pattern." They should still be able to show where Δ^2 comes from.

Part 3: Newton's method

3a.

> Under that linear approximation (the slope starts at $\mathrm{NLL}'(\lambda)$ and changes at a constant rate $\mathrm{NLL}''(\lambda)$), find the value λ^* where the slope reaches zero. Write λ^* in terms of λ, $\mathrm{NLL}'(\lambda)$, and $\mathrm{NLL}''(\lambda)$.

Answer.

$$\lambda^* = \lambda - \frac{\mathrm{NLL}'(\lambda)}{\mathrm{NLL}''(\lambda)}$$

Why. A straight line through the point $(\lambda, \mathrm{NLL}'(\lambda))$ with slope $\mathrm{NLL}''(\lambda)$ is $\mathrm{NLL}'(\lambda) + \mathrm{NLL}''(\lambda)(\lambda^* - \lambda)$. Set it to zero and solve for λ^*. Equivalently, "rise over run": the line has to drop by $\mathrm{NLL}'(\lambda)$ at a rate of $\mathrm{NLL}''(\lambda)$ per unit, which takes a run of $\mathrm{NLL}'/\mathrm{NLL}''$, so $\lambda^* = \lambda - \mathrm{NLL}'/\mathrm{NLL}''$. Dividing by the curvature sets the step size: strong curvature means a small step, weak curvature a big one.

Watch for. A plus sign instead of a minus (the step must go downhill: if the slope is negative, λ should increase). Writing the line through the origin, $\mathrm{NLL}''(\lambda)\,\lambda^* = 0$, instead of through the current point. Students who know the tangent-line form $y - y_0 = m(x - x_0)$ from Calc I get there fastest; for others, a sketch of NLL' against λ with the straight line through the current point usually unlocks it. In the debrief, point out that a straight-line slope is the same as a parabola-shaped NLL, so Newton's method is jumping to the bottom of that parabola.

3b.

> Start at $\lambda = 2$ and take three Newton steps, using your exact derivatives from 1a.

Answer.

Step	Guess	Slope, NLL′	Curvature, NLL″	Next guess
1	2.0000	-3.0000	4.5000	2.6667
2	2.6667	-0.7500	2.5312	2.9630
3	2.9630	-0.0750	2.0503	2.9995

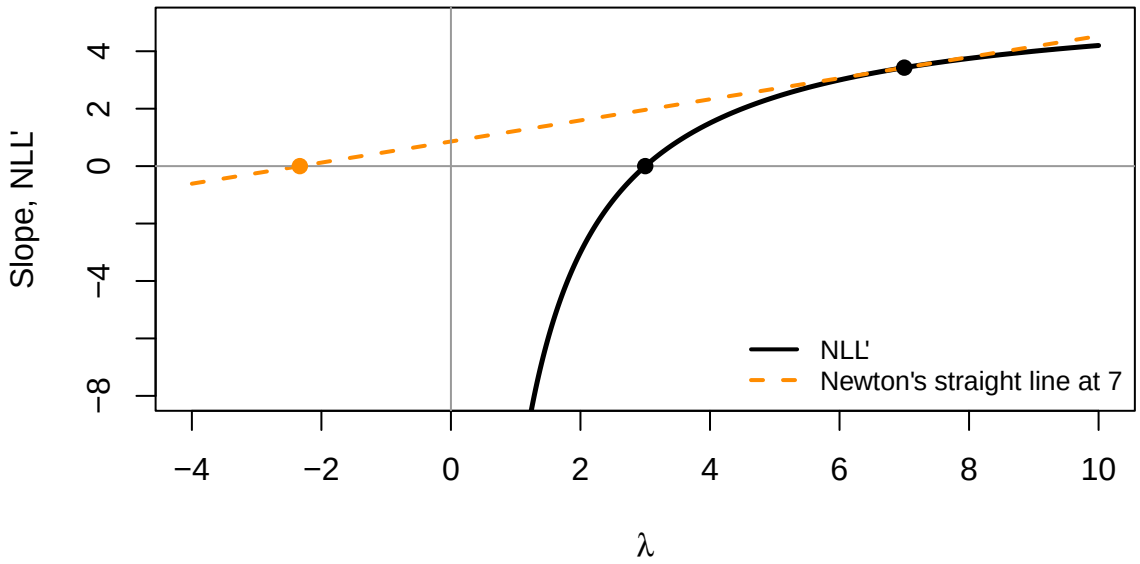
Why. The distance to the MLE goes 1, 0.33, 0.037, 0.0005: each error is roughly the square of the one before. That fast ("quadratic") convergence near the answer is why derivative-based methods are popular.

Watch for. Rounding λ to 2.67 and carrying the rounding error forward (still converges, just more slowly). Plugging the new guess into the old curvature.

3c.

Now start at $\lambda = 7$ and take one step. What went wrong? Sketch $\text{NLL}'(\lambda)$ and the straight line Newton's method used at $\lambda = 7$ to explain why, and suggest one way to fix it.

Answer. $\text{NLL}'(7) = 3.429$, $\text{NLL}''(7) = 0.367$, so the next guess is $7 - 3.429/0.367 = -2.333$. A Poisson mean can't be negative, and the NLL isn't even defined there.



Why. Far from the MLE the slope NLL′ levels off (small curvature), so the straight line Newton's method draws at 7 is very shallow and doesn't cross zero until $\lambda = -2.33$, outside the allowed range. (The true slope reaches zero at 3.) In general the update here simplifies to $\lambda_{\text{new}} = 2\lambda - \lambda^2/3$, which is negative for any start above 6.

Acceptable fixes (any one):

- **Shorter steps.** If a step makes the NLL worse or leaves the allowed range, halve it and try again ("backtracking").
- **Better starting values.** Start near a sensible guess, such as the sample mean.
- **Constraints.** Use a method that respects bounds (e.g. `method = "L-BFGS-B"` with `lower = 0.001`).
- **Fit on the log scale.** Optimize over $\theta = \log \lambda$, so every value of θ gives a positive λ. Newton's method on θ from $\lambda = 7$ gives $\lambda = 3.953, 3.106, 3.002, 3.000$: no overshoot. (This one comes up in Wednesday's lecture.)

Watch for. Concluding Newton's method is "broken." It works well near the answer and badly far from it, which is why starting values matter for every optimizer.

3d.

(If you have time.) Real optimizers usually don't know the exact derivatives and use finite-difference estimates instead. By hand, take one Newton step from $\lambda = 2$ using your estimates from 2a and 2b in place of the exact derivatives. How close is it to your first step in 3b? Based on 2a, how could you make it closer?

Answer. $2 - (-2.0332)/4.6464 = 2.4376$, versus 2.6667 with exact derivatives: the right direction, but a shorter step. To do better, use a smaller Δ, or average the forward and backward differences from 2b (the same as the chord from 1.5 to 2.5, centered on 2, slope -3.1948), which gives 2.6876.

Why. The forward slope belongs to 2.25, not 2 (2a), so it understates how steep the NLL is at 2. Errors in the slope and curvature change where each step lands, not where the method stops ($\text{NLL}' = 0$), so the method still converges. `optim()` uses a much smaller step ($\Delta = 0.001$ by default), so its estimates are far closer.

Watch for. Reusing the exact slope from 1a with the estimated curvature (that mixes the two methods).